



# **Hantro SW VC9000D**

## **Release Notes**

Nov. 2024

## Legal Notes

---

### **COPYRIGHT INFORMATION**

This document contains proprietary information of VeriSilicon Corporation. VeriSilicon Corporation reserves the right to make changes to any products herein at any time without notice. VeriSilicon Corporation does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon Corporation; nor does the purchase or use of a product from VeriSilicon Corporation convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon Corporation or third parties.

### **DISCLOSURE/RE-DISTRIBUTION LIMITATIONS**

This document contains proprietary information of VeriSilicon Corporation. The information contained in this VeriSilicon Confidential document may be directly re-distributed or may be adapted for re-distribution by VeriSilicon customer SoC vendors who have licensed the related core IP, with the understanding that re-distribution is limited to those customers of the SoC vendor who have active and valid NDA or licenses applicable for the SoC which contains the related VeriSilicon core IP. Otherwise, the information contained herein is not to be used by or disclosed to the third parties without the express written permission of an officer of VeriSilicon Corporation.

(VeriSilicon Distribution **Level B CONFIDENTIAL – DISTRIBUTION RESTRICTED**).

Upon request VeriSilicon provides this document in Word format for adaptive inclusion in documentation intended for your customers. VeriSilicon requests a pre-release copy of the adaptation for review and approval. In the adapted document please include a reference to the copyright owner, for example: "This chapter contains copyright confidential and proprietary material disclosed with permission of VeriSilicon Corporation. VeriSilicon has authorized re-distribution of this material restricted to those NDA partners and licensees of *[the SoC vendor]* who are engineering *[SoC vendor's product]* which includes the discussed VeriSilicon IP."

### **TRADEMARK ACKNOWLEDGMENT**

VeriSilicon Corporation and the VeriSilicon Corporation logo design are trademarks or registered trademarks of VeriSilicon Corporation. All other brand and product names may be trademarks of their respective companies.

Copyright © 2024 by VeriSilicon Corporation. All rights reserved.

## Table of Contents

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| <b>Legal Notes.....</b>                                            | <b>2</b> |
| <b>Table of Contents.....</b>                                      | <b>3</b> |
| <b>1 General Information .....</b>                                 | <b>4</b> |
| <b>2 Package Contents List .....</b>                               | <b>4</b> |
| <b>3 Package Structure.....</b>                                    | <b>5</b> |
| 3.1 The main structure of the VC9000D_Cmodel_bin package .....     | 5        |
| 3.2 The main structure of the VC9000D_RV_3rdParty package .....    | 5        |
| 3.3 The main structure of the VC9000D_Ctrlsw_Src package .....     | 5        |
| 3.4 The main structure of the VC9000D_Kernel package .....         | 6        |
| 3.5 The main structure of the example test package .....           | 6        |
| <b>4 Package Assemble(MUST).....</b>                               | <b>7</b> |
| <b>5 Third-party source code reference .....</b>                   | <b>7</b> |
| <b>6 How to load pcie driver.....</b>                              | <b>7</b> |
| <b>7 How to compile VC9000D pcie test binary in FPGA ENV .....</b> | <b>8</b> |
| 7.1 0x1FF0.....                                                    | 8        |
| 7.2 0x201B .....                                                   | 8        |
| <b>8 How to run the binaries .....</b>                             | <b>9</b> |
| 8.1 For CModel.....                                                | 9        |
| 8.2 For FPGA.....                                                  | 9        |

## 1 General Information

This guide provides an overview of the VC9000D SW package, including some example binary running command lines and sample compile commands, for customer to kick off the sw package quickly.

If need more details, please refer to the documents listed in Chapter 2 "Documents List".

**Version:** 2.4.85

**Release Package Type:** CtrlSW\_Src

**Release Formats:**

hevc/h264/jpegdec/avsdec/mpeg2dec/mpeg4dec/rvdec/vc1dec/vp6dec/vp8dec/avs2/vp9/av1/ppdec

**CModel Built Environment:**

ldd version: ldd (Ubuntu GLIBC 2.23-0ubuntu10) 2.23

Linux version: 4.4.0-62-generic (buildd@lcy01-30)

gcc version: 5.4.0 20160609 (Ubuntu 5.4.0-6ubuntu1~16.04.4)

## 2 Package Contents List

- |—— doc
  - | |—— Hantro.VC9000D.Series.Video.Decoder.FPGA.Verification.Guide-v1.02-D-20231229.pdf
  - | |—— Hantro.VC9000D.Video.Decoder.API.User.Manual-v2.4.80-A-20240103.pdf
  - | |—— Hantro.VC9000D.Video.Decoder.TestBench.User.Manual-v2.4.80-A-20240103.pdf
  - | |—— Hantro.VC9000D.Video.Decoder.Wrapper.Layer.API.User.Manual-v2.4.76-A-20240328.pdf
  - | |—— license
    - | |—— Hantro.Programming.Memo.VPU.Decoder.Software.Source.License.Summary-v0.80-D-20240515.pdf
- |—— Hantro\_SW\_Decoder\_Cmodel\_Bin\_Proprietary\_v.2.4.85\_CL1022686.tgz
- |—— Hantro\_SW\_Decoder\_C\_Test\_RV\_3rdParty\_rm\_parser\_Proprietary\_v.2.4.85\_CL1022686.tgz
- |—— Hantro\_SW\_Decoder\_CtrlSW\_Proprietary\_v.2.4.85\_CL1022686P.tgz
- |—— Hantro\_SW\_Decoder\_Example\_test\_commands\_Proprietary\_v2.4.85\_CL1022686.tgz
- |—— Hantro\_SW\_Decoder\_Kernel\_Redistributable\_v.2.4.85\_CL1022686P.tgz

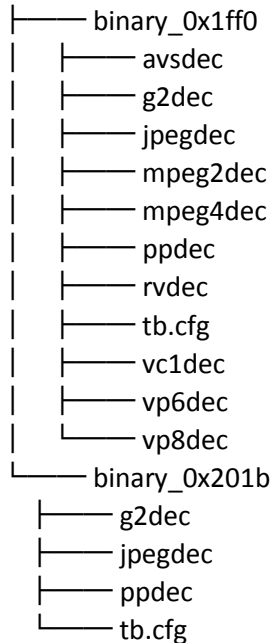
2 directories, 10 files

### 3 Package Structure

#### 3.1 The main structure of the VC9000D\_Cmodel\_bin package

Hantro\_SW\_Decoder\_Cmodel\_Bin\_Proprietary\_v.2.4.85\_CL1022686.tgz. It's the main structure for vc9000d cmodel binaries.

VC9000D\_CtrlSW\_V2.4.85/

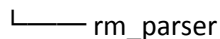


**Binary\_0x1ff0/0x201b** CModel binaries and tb.cfg (configuration file)

#### 3.2 The main structure of the VC9000D\_RV\_3rdParty package

Hantro\_SW\_Decoder\_C\_Test\_RV\_3rdParty\_rm\_parser\_Proprietary\_v.2.4.85\_CL1022686.tgz. It's about the rv source code.

VC9000D\_CtrlSW\_V2.4.85/software/test/rv

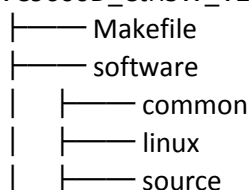


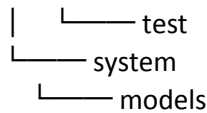
**rm\_parser** Decoder C test rv 3rdParty rm\_parser

#### 3.3 The main structure of the VC9000D\_Ctrlsw\_Src package

Hantro\_SW\_Decoder\_CtrlSW\_Proprietary\_v.2.4.85\_CL1022686P.tgz. It's the main structure for vc9000d ctrlsw, including cmodel libs and user space code.

VC9000D\_CtrlSW\_V2.4.85/



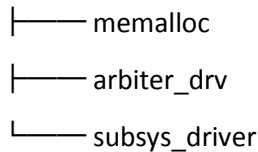


|                 |                                                                                                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Makefile</b> | a unified binary compilation files, and includes some global common defines                                                                                             |
| <b>software</b> | including sw user space code                                                                                                                                            |
| <b>--common</b> | including the analysis of various binary compilation options                                                                                                            |
| <b>--linux</b>  | the code of DWL layer                                                                                                                                                   |
| <b>--source</b> | a folder for storing API code of each format                                                                                                                            |
| <b>--test</b>   | the path where testbench is located, and the compilation binary will select the corresponding testbenches in the folder to compile according to the compilation target. |
| <b>system</b>   | header files about cmodel emulation                                                                                                                                     |

### 3.4 The main structure of the VC9000D\_Kernel package

**Hantro\_SW\_Decoder\_Kernel\_Redistributable\_v2.4.85\_CL1022686P.tgz.** It's about the kernel driver source code.

VC9000D\_CtrlSW\_V2.4.85/software/linux

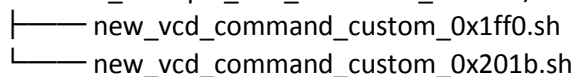


|                      |                                 |
|----------------------|---------------------------------|
| <b>memalloc</b>      | memory allocation kernel driver |
| <b>arbiter_drv</b>   | arbiter kernel driver           |
| <b>Subsys_driver</b> | linux kernel driver code        |

### 3.5 The main structure of the example test package

**Hantro\_SW\_Decoder\_Example\_test\_commands\_Proprietary\_v2.4.85\_CL1022686.tgz.** It's about the running command for customer reference.

VC9000D\_Example\_test\_command\_V2.4.85/



|                                                |                                       |
|------------------------------------------------|---------------------------------------|
| <b>new_vcd_command_custom_0x1ff0/0x201b.sh</b> | VC9000D Regular Example test commands |
|------------------------------------------------|---------------------------------------|

## 4 Package Assemble(MUST)

---

Please untar the 3.2 VC9000D\_RV\_3rdParty package and 3.3 VC9000D\_Ctrlsw\_Src package and 3.4 VC9000D\_Kernel package, and then

the structure of [VC9000D assembled path]/software/linux path will like below.

**This step is necessary before doing below compile steps!!!**

linux

```
|—— arbiter_drv
|—— dwl
|—— memalloc
|—— mpeg4
|—— pp
|—— subsys_driver
```

## 5 Third-party source code reference

---

### Nestegg

You can get the Nestegg code from <https://github.com/mozilla/nestegg>

### How to compile nestegg in X86 environment.

```
$> cd [3rd-party path]/nestegg-master
```

```
$> autoreconf --install
```

```
$> ./configure --prefix=${PATH of compiled nestegg} --enable-shared CFLAGS=-fPIC
```

```
$> make && make install
```

## 6 How to load pcie driver

---

```
$> cd [VC9000D assembled path]/software/linux/arbiter_drv
```

```
$> make clean all
```

```
$> ./arbiter_load.sh
```

```
$> cd [VC9000D assembled path]/software/linux/arbiter_drv/test
```

```
$> make clean all
```

```
$> ./arbiter_test [option]
```

#### Example ####

sc:

```
$> cd [VC9000D assembled path]/software/linux/arbiter_drv/test
```

```
$> make clean all
```

```
$> ./arbiter_test --arbId=0 --arbEnable=1 --timeWindow=29
```

```
$> ./arbiter_test --arbId=0 --masterId=0 --onLine=1 --urgent=1 --weight=31 --bwOverflow=0
```

mc:

```

$> cd [VC9000D assembled path]/software/linux/arbiter_drv/test

$> make clean all

$> ./arbiter_test --arbid=0 --arbEnable=1 --timeWindow=29

$> ./arbiter_test --arbid=0 --masterId=0 --onLine=1 --urgent=1 --weight=31 --bwOverflow=0

$> make clean all

$> ./arbiter_test --arbid=1 --arbEnable=1 --timeWindow=29

$> ./arbiter_test --arbid=1 --masterId=0 --onLine=1 --urgent=1 --weight=31 --bwOverflow=0

#####

$> cd [VC9000D assembled path]/software/linux/subsys_driver
$> make clean && make SUPPORT_MMU=y SUPPORT_48PA_MMU=y SUPPORT_AXIFE=y
$> ./driver_load.sh
$> cd [VC9000D assembled path]/software/linux/memalloc
$> make clean && make
$> ./memalloc_load.sh

```

## 7 How to compile VC9000D pcie test binary in FPGA ENV

Use g2dec as reference, and it's similar to other formats.

### 7.1 0x1FF0

#### 7.1.1 For normal(Lowlatency and multi-core are used same commands)

```

$> cd [VC9000D assembled path]
$> make clean g2dec ENV=x86_linux_pci RELEASE=y ENABLE_FPGA_VERIFICATION=y
USE_HW_PIC_DIMENSIONS=y CLEAR_OUT_BUFFER=y USE_64BIT_ENV=y WEBM_ENABLED=y
NESTEGG=[ path of nestegg] USE_VCMD=y SUPPORT_MMU=y SUPPORT_AXIFE=y
SUPPORT_48PA_MMU=y USE_PPU_V9_2_3=n DEF_HWBUILD_ID=1FF0

```

#### 7.1.2 For dec400

```

$> cd [VC9000D assembled path]
$> make clean g2dec ENV=x86_linux_pci RELEASE=y ENABLE_FPGA_VERIFICATION=y
USE_HW_PIC_DIMENSIONS=y CLEAR_OUT_BUFFER=y USE_64BIT_ENV=y SUPPORT_DEC400=y
WEBM_ENABLED=y NESTEGG=[ path of nestegg] USE_VCMD=y SUPPORT_MMU=y
SUPPORT_AXIFE=y SUPPORT_48PA_MMU=y USE_PPU_V9_2_3=n DEF_HWBUILD_ID=1FF0

```

### 7.2 0x201B

Normal, lowlatency and dec400 are used same commands

```

$> cd [VC9000D assembled path]
$> make clean g2dec ENV=x86_linux_pci RELEASE=y ENABLE_FPGA_VERIFICATION=y
USE_HW_PIC_DIMENSIONS=y CLEAR_OUT_BUFFER=y USE_64BIT_ENV=y WEBM_ENABLED=y
NESTEGG=[ path of nestegg] USE_VCMD=y SUPPORT_MMU=y SUPPORT_AXIFE=y
SUPPORT_48PA_MMU=y USE_PPU_V9_2_3=y DEF_HWBUILD_ID=201B

```

Then you will find pcie test binary in out/x86\_linux\_pci/release.

## 8 How to run the binaries

Here are some sample command lines for reference(Please keep tb.cfg & binary in same path when customers run cases)

### 8.1 For CModel

- a) **hevc/h264/av1/vp9/avs2 use the shared g2dec**  
`$> ./g2dec -d1 -N10 -Ooutput.yuv input_stream`  
 #-d1: output raster scan data (no scaling)  
 #-N10: Stop after 10 output pictures
- b) **jpeg (VC1,MPEG2,VP8,VP6,AVS,RV formats are the similar, just replace the binary name)**  
`$> ./jpegdec -Cx8 -Cy8 -Cw144 -Ch144 -D72x72 -Ooutput input_stream`  
 #-Cxywh: Support flexible crop; crop from (x,y) with cropped size (width x height)  
 #-Cx8 -Cy8: crop from (8,8)  
 #-Cw144 -Ch144: cropped size 144x144  
 #-D72x72: Support flexible scale; 72x72 specifies the width and height of the post-processor output
- c) **pp standalone**  
`$> ./ppdec -H168 -W1744 -S1744 -Oout.yuv in.yuv`  
 #-S1744: Set input stream stride to 1744 bytes  
 #-W1744: Set input stream width to 1744 pixels (MUST)  
 #-H168: Set input stream height to 168 pixels (MUST)
- d) **MPEG4/DIVX3/4/5/6/Sorenson used the shared mpeg4dec**  
**MPEG4**  
`$> ./mpeg4dec -m -Ooutput input_stream`  
 #-m: Write MD5 sum for each picture to output instead of YUV  
**DIVX3**  
`$> ./mpeg4dec --custom 320x240 -N20 -Ooutput input_stream`  
 #--custom 320x240: Decode DivX3 stream of resolution 320x240(widthxheight)(MUST)  
**DIVX4/5**  
`$> ./mpeg4dec --strm-custom -D144x144 -Ooutput input_stream`  
 #--strm-custom: Decode DivX4 or DivX5 stream.(MUST)  
**Sorenson**  
`$> ./mpeg4dec --strm-sorenson -d1 -Ooutput input_stream`  
 #--strm-sorenson: Decode Sorenson Spark stream(MUST)

### 8.2 For FPGA

- a) **normal**  
 Please refer to CModel command lines.
- b) **lowlatency**  
`$> ./g2dec -L --cr-first -d1 -Ooutput input_stream`  
 #-L: Enable low latency platform running flag(MUST)  
 #--cr-first: PP outputs chroma in CrCb order
- c) **multi-core**  
`$> ./g2dec --mc -d1 -Ooutput input_stream`  
 #--mc: Enable multi-core decoding(MUST)
- d) **dec400**  
`$> ./g2dec --pp-comp -A256 -N10 --pp=0 --pp-comp -f -d1 -Ooutput input_stream`

# --pp-comp: Enable dec400 decoding(MUST)

More details about parameters for different functions please run "g2dec -H", or refer to doc/Hantro.VC9000D.Video.Decoder.TestBench.User.Manual-v2.4.80-A-20240103.pdf.